Using Zero-Knowledge to Reconcile Law Enforcement Secrecy and Fair Trial Rights in Criminal Cases

Dor Bitan*
University of California at Berkeley

Ran Canetti†
Boston University

Shafi Goldwasser‡
University of California at Berkeley

Rebecca Wexler§
University of California at Berkeley

ABSTRACT

The use of hidden investigative software to collect evidence of crimes presents courts with a recurring dilemma: On the one hand, there is often clear public interest in keeping the software hidden to preserve its effectiveness in fighting crimes. On the other hand, criminal defendants have rights to inspect and challenge the full evidence against them, including law enforcement's investigative methods. In fact, in the U.S. adversarial legal system, the defendant's rights to scrutinize the government's tools are crucial to the truth-seeking process and to keeping law enforcement conduct lawful and constitutional. Presently, courts balance these conflicting interests on a case-by-case basis through evidentiary privilege law, often voicing their frustration with the challenging dilemma they face.

We demonstrate how judicious use of a sophisticated cryptographic tool called Zero Knowledge Proofs (ZKPs) could help to mitigate this dilemma: Based on actual court cases where evidence was collected using a modified version of a peer-to-peer software, we demonstrate how law enforcement could, in these cases, augment their investigative software with a ZKP-based mechanism that would allow them to later provide full responses to challenges made by a defense expert – and allow a defense expert to independently verify law enforcement claims – *while keeping the software hidden.*

We demonstrate the technical feasibility of our mechanism via a proof-of-concept implementation. We also propose legal analysis that justifies its use, discusses its merits, and considers the legal implications that the very existence of such a mechanism might have, even in cases where it has not been used. Our proof-of-concept may also extend to other verification dilemmas in the legal landscape.

CCS CONCEPTS

• **Theory of computation → Cryptographic protocols**; • **Applied computing → Law**.

KEYWORDS

Zero-knowledge proofs, Law enforcement evidentiary privilege, Fair trial rights

ACM Reference Format:
Dor Bitan, Ran Canetti, Shafi Goldwasser, and Rebecca Wexler. 2022. Using Zero-Knowledge to Reconcile Law Enforcement Secrecy and Fair Trial Rights in Criminal Cases. In *Proceedings of the 2022 Symposium on Computer Science and Law (CSLAW '22), November 1–2, 2022, Washington, DC, USA.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3511265.3550452

1 INTRODUCTION

In the federal court case of *U.S. v. Michaud*, a criminal defendant charged with receiving Child Sexual Abuse Materials (CSAM) over the Internet sought to review the source code for a software that law enforcement had used to investigate his online activity.[1] The defense argued that it needed the code to determine the lawfulness, constitutionality, and accuracy of law enforcement's use of the investigative tool.[2] The court agreed, emphasizing that the defense was "not required to accept the government's assurances," but rather was entitled to review the code directly to determine for itself how the software worked.[3] On the other side, the prosecution[4] argued that disclosure could "allow individuals to devise measures to counteract these [investigative] techniques in order to evade detection[.]"[5] As a result, the prosecution argued that it should be entitled to bring the case to trial without disclosing the code to a defense expert witness.[6] The prosecution submitted evidence for *ex parte* and *in camera* review by the judge alone in chambers, ultimately persuading him that the code was too sensitive to be disclosed.[7] Faced with this conflict, the court lamented: "What

*Postdoctoral researcher, Simons Institute for the Theory of Computing, UC Berkeley. Email: bitandor@berkeley.edu.
†Professor of Computer Science, Boston University; Director, Center for Reliable Information System and Cyber Security.
‡Director of the Simons Institute for the Theory of Computing, and C. Lester Hogan Professor in Electrical Engineering and Computer Sciences, UC Berkeley; RSA Professor of Electrical Engineering and Computer Science, MIT.
§Assistant Professor of Law, UC Berkeley; Faculty Co-Director, Berkeley Center for Law and Technology.

This material is based upon work supported by DARPA under Agreement No. HR00112020021. Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the United States Government or DARPA..

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CSLAW '22, November 1–2, 2022, Washington, DC, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9234-1/22/11…$15.00
https://doi.org/10.1145/3511265.3550452

[1] Order, U.S. v. Michaud, 15-cr-5351, ECF 205 (W.D. Wash., May 18, 2016).
[2] Motion to Compel, U.S. v. Michaud, 15-cr-5351, ECF 54, at *1-2 (W.D. Wash., Nov. 4, 2015).
[3] Order, U.S. v. Michaud, 15-cr-5351, ECF 205, at *4 (W.D. Wash., May 18, 2016).
[4] This paper sometimes uses the phrase "law enforcement" to encompass both police and prosecutors, and other times identifies prosecutors separately.
[5] Response, U.S. v. Michaud, 15-cr-5351, ECF 74, at *15 (W.D. Wash., Dec. 4, 2015).
[6] Reply, U.S. v. Michaud, 15-cr-5351, ECF 149, at *2-3 (W.D. Wash., Feb. 8, 2016).
[7] Order, U.S. v. Michaud, 15-cr-5351, ECF 205, at *3 (W.D. Wash., May 18, 2016).

should be done about it when, under these facts, the defense has a justifiable need for information in the hands of the government, but the government has a justifiable right not to turn the information over to the defense?"[8]

The *Michaud* court's dilemma is not unique. Similar dilemmas involving hidden investigative software used by law enforcement (e.g. [10, 38, 41, 53]) occur repeatedly in criminal cases and have long provoked heated debate by courts, legal scholars, and the public alike [5, 34, 39, 47]. The problem is challenging for multiple reasons, not least of which is the risk of bad faith conduct by either defense counsel or law enforcement. On the one hand, criminal defense arguments for accessing sensitive information about law enforcement technology might be fishing expeditions designed to graymail the prosecution into dropping criminal charges. On the other hand, law enforcement assertions about the value of secrecy, or about the functional characteristics of an investigative tool, could be exaggerated or mistaken, or even pretextual attempts to conceal negligent or wrongful conduct. Meanwhile, judges trying to weigh the conflicting interests in these types of high stakes scenarios can suffer from expertise deficits about both security and auditing needs.

Furthermore, judicious resolution of such dilemmas requires the ability to correctly assess both the potential harms of disclosure, and the potential harms of shielding the software from scrutiny, as pertaining to the specific case. However, the process by which courts typically attain such technical knowledge — namely, the adversarial process whereby domain experts on both sides educate the court — is disabled by the very claims that this process is expected to help adjudicate. This well-documented situation often leaves courts in a bind [46].

But is this dilemma inherent? What if there were a technical solution that could enable the parties to engage in full adversarial scrutiny of the law enforcement software tools, without jeopardizing their future efficacy?

This article proposes that, in criminal cases that involve hidden investigative software, astute use of a powerful computer science tool, Zero-Knowledge Proofs (hereafter ZKPs), can help resolve the recurring conflict between preserving the secrecy of law enforcement investigative tools and preserving defendants' fair trial rights. ZKPs are a family of cryptographic tools, first suggested by Goldwasser, Micali, and Rackoff in [24], and then shown to be universally applicable by Goldreich, Micali, and Wigderson [22]. ZKPs enable one party to prove to another party mathematical claims regarding a secret digital object without disclosing that object. While using ZKPs in a variety of legal settings has been proposed in the past (e.g.[3, 35, 36]), to the best of our knowledge no concrete proposal has been made to date for using ZKPs as a tool for resolving the dilemma at hand. This work demonstrates how ZKPs can enable a defense expert witness to independently verify relevant details about the structure and operation of hidden investigative software, while simultaneously enabling law enforcement to keep the secrecy of sensitive information (such as full source code or specific deployment details) that, if leaked, might undermine the efficacy of the software for future investigations. Our proposals also extend to cases where the investigative software was procured by law enforcement from a third party vendor.

[8] Order, U.S. v. Michaud, 15-cr-5351, ECF 205, at *5 (W.D. Wash., May 18, 2016).

1.1 Background

The legal baseline. The U.S. criminal legal system uses an *adversarial* process to seek truth. This means that law enforcement is responsible for investigating evidence of guilt, while defense counsel is responsible for investigating evidence of innocence and for identifying flaws or weaknesses in the government's case [55].

Evidentiary privileges impede this adversarial process by suppressing relevant evidence from the jury, judge, and opposing parties. In general, the costs that privileges impose on truth-seeking are justified by countervailing benefits to societal policy goals that are extrinsic to the judicial fact-finding system [32].

Currently, courts in hidden investigative software cases resolve the tension between law enforcement secrecy and fair trial rights by applying a qualified, common-law evidentiary privilege called *the law enforcement privilege*.[9] Although the procedures vary by jurisdiction, a law enforcement entity claiming the privilege generally must show that the allegedly privileged information concerns confidential investigative techniques or procedures, the broad dissemination of which might enable future criminals to evade detection.[10] Next, the court will balance law enforcement's interest in secrecy against a criminal defendant's interest in disclosure.[11] In the *Michaud* case, the court found that the government had a valid claim to the law enforcement privilege, but that upholding the privilege violated the defendant's right to scrutinize the evidence.[12] As a result, the court suppressed all evidence derived from the government's use of the hidden investigative software,[13] ultimately leading the prosecution to seek dismissal of the criminal charges rather than disclose the code and get the evidence admitted.[14] However, most courts facing similar conflicts have instead decided that a law enforcement affidavit attesting to the functional characteristics of a hidden investigative software program suffice to satisfy fair trial rights.[15] Those courts have permitted prosecutions to proceed without requiring disclosures of sensitive information that would enable a criminal defense expert witness to independently verify law enforcement claims about how particular hidden investigative software tools work.[16]

Either of these existing outcomes has drawbacks. Even if all parties are acting in good faith, a court's decision to prioritize either criminal defense rights or law enforcement secrecy will necessarily undermine the important conflicting values represented by the losing party's position. A court's decision to order disclosure risks damaging the efficacy of what may be a lawful, constitutional, and highly valuable tool. Alternatively, should the prosecution choose dismissal rather than comply with such a disclosure order, such decision risks allowing the alleged perpetrators of what may be

[9] *See*, Kenneth W. Graham, Jr. and Ann Murphy, Fed. Prac. Proc. § 5662 (1st ed.); Edward J. Imwinkelried, The New Wigmore: A Treatise on Evidence § 7.4.2 (3d ed.).
[10] *See, e.g.*, Black v. Sheraton, 564 F.2d 531, 545-46 (D.C. Cir. 1977).
[11] *Id.* at 547.
[12] Order, U.S. v. Michaud, 15-cr-5351, ECF 205, at *5 (W.D. Wash., May 18, 2016).
[13] Order, U.S. v. Michaud, 15-cr-5351, ECF 212, at *1 (W.D. Wash., May 25, 2016).
[14] Motion to Dismiss, U.S. v. Michaud, 15-cr-5351, ECF 227 (W.D. Wash., March 3, 2017).
[15] *See, e.g.*, U.S. v. Pirosko, 787 F.3d 358, 365 (6th Cir. 2015) (citing cases).
[16] *Id.*

terrible crimes to return unpunished to the community where they may recidivate and harm future victims.[17]

At the same time, a court's decision to permit a prosecution to proceed on the basis of a law enforcement officer's affidavit alone, without ordering disclosure of potentially relevant exculpatory or impeachment information to the defense, undermines defense counsel's ability to provide contrary evidence. As Supreme Court Justice William Douglas said in the related context of the confidential informer's identity privilege, unless courts order disclosure in these types of circumstances, "the policeman himself conclusively determines the validity of his own arrest."[18] Ultimately, since the U.S. criminal legal system depends on adversarial scrutiny to seek truth, blocking defense access obstructs the truth-seeking process of the courts [55]. Furthermore, repeated deference to law enforcement claims of privilege creates public unease and distrust of the judicial system, even by organizations that are naturally sympathetic to the protection of victims from criminal abuse.[19]

Investigating CSAM distribution over P2P networks. We turn attention to two cases that have directly inspired this work: *U.S. v. Budziak* and *U.S. v. Clements*. Both cases are centered around peer-to-peer (hereafter P2P) networks. P2P networks enable users to share documents directly, with no intermediary involved [44, 51]. Furthermore, each document is partitioned into multiple shares, where each share is stored by multiple network members. Similarly, to retrieve ("download") a document from the network, the P2P software typically assembles the document from shares held by different members of the network. While the decentralized nature of P2P networks is beneficial for resilience to censorship as well as anonymity for whistle-blowers, journalists, and news organizations [42], it is unfortunately also conducive to illegal activity, with a primary example being the distribution of CSAM [29, 30, 37, 43, 54]. Indeed, P2P networks provide distributors of CSAM (who are often also involved in offline forms of child abuse [10]) with a cloak under which they can perform their illegal activity.

To investigate distribution of CSAM over P2P networks, law enforcement sometimes uses hidden software that mimics the P2P application in use, but includes some modifications. The stated reason for using such modified software, rather than simply joining the P2P network using the publicly-available application, is to enable law enforcement to target a specific device and make sure that all the pieces of a given document originate from that device. Performing a single-source download of the document provides stronger evidence that the operator of the targeted device knowingly possessed and distributed said document. It is also instrumental for law enforcement's showing of probable cause to seize and search the relevant device.

The Budziak Case. The *Budziak* prosecution charged the defendant with receiving and distributing CSAM via P2P software.[20]

Here, the hidden software was built by a law enforcement contractor and connected to the LimeWire P2P network. Following successful downloads of CSAM material from a device in Mr. Budziak's residence, law enforcement obtained a search warrant and seized a number of devices from Mr. Budziak's residence that contained a substantial quantity of CSAM material.[21]

At the discovery stage of the case, the defense sought to compel the government to disclose an executable copy, technical specifications, and documentation for the software, citing the defendant's testimony that the documents downloaded by the hidden software were set to be un-shareable, along with specific concerns that the software might have included a known exploit that was widely used to surreptitiously mark files on the target device as shareable without the knowledge of the device owner. (Apparently, the exploit was created to increase the sharing of files in order to boost the use of LimeWire.) The prosecution argued that the information was protected by the law enforcement evidentiary privilege. The trial proceeded without disclosure and resulted in a conviction for possession *and distribution* of CSAM.[22] On appeal, the Ninth Circuit held that the defendant was entitled to the undisclosed information and remanded to the district court, which, in turn, ordered the prosecution to disclose both executable and source code versions of the software program.[23] A month later, the prosecution clarified that the software was not actually created by the FBI (as previously claimed) but rather "by a private company under contract with the FBI."[24] Moreover, the prosecution explained that it could not comply with the district court's disclosure order because the FBI lacked access to the source code.[25] The FBI's inability to comply with the disclosure order ultimately led to a dismissal of criminal charges in the case.[26] (While the Ninth Circuit opinion in *Budziak* has been widely cited, this work is the first to identify, via original research into PACER court filings, the post-remand failure of the FBI to disclose either the source code or an executable copy of the program.)

The Clements Case. Similar to *Budziak*, the *Clements* prosecution charged the defendant with receiving and distributing CSAM via P2P software.[27] Here, however, the modified version of the P2P software was developed and owned by a private company, TLO, LLC, and licensed to law enforcement.[28] Furthermore, here the results generated by law enforcement's use of the TLO software formed the sole evidence of guilt supporting the receipt and distribution charges.[29] The defense sought to test an executable copy of the software "to determine the validity and/or reliability

[17] Indeed, law enforcement has emphasized the possible risks from disclosure by arguing that hidden investigative software is "revolutionary" and "valuable" and helps "bring to justice the most prolific offenders." Motion, United States v. Ocasio, 11-cr-2728, ECF 148-3, at *12 (W.D. Tex. Nov. 9, 2011).

[18] McCray v. Illinois, 386 U.S. 300, 315-16 (1967) (Douglas, *J.*, dissenting).

[19] Letter from Sarah St. Vincent, Human Rights Watch, to Dep't of Justice (Feb. 1, 2019), https://www.hrw.org/sites/default/files/supporting_resources/hrw_ltr_to_doj.pdf.

[20] U.S. v. Budziak, 697 F.3d 1105, 1112 (9th Cir. 2012).

[21] *Id.* at 1107.

[22] Possession of CSAM is punishable by up to 10 years imprisonment, whereas distribution of CSAM carries a significantly heavier punishment with a mandatory minimum of 5 years and maximum of 20 years imprisonment. 18 U.S.C. § 2252.

[23] Order, U.S. v. Budziak, 5-cr-284, ECF 248, at *1 (N.D. Cal. Feb. 18, 2014).

[24] U.S. v. Budziak, 5-cr-284, ECF 251, at *4 (N.D. Cal. May 1, 2014).

[25] In the FBI's own words, it had searched for the code by: "contacting th[e] contractor to determine if they still retained a copy of the source code. Most of the contractor's personnel involved in the development effort are no longer with the company. The FBI located some of the contractor's former personnel and contacted them to determine if they still possessed a copy of the source code. These efforts were unsuccessful." *Id.*

[26] *Id.*

[27] Indictment, U.S. v. Clements, 15-cr-275, ECF 1, at *1 (N.D. Ohio July 21, 2015).

[28] Motion to Compel, U.S. v. Clements, 15-cr-275, ECF 17, at *5 (N.D. Ohio January 18, 2016).

[29] *Id.* at *9 ("The purported downloads form the entire basis for the receipt and distribution offense set forth in Count 1 of the Indictment.").

of the Government's forensic evidence . . . as well as to determine whether the software conducted a search of Clements's computer beyond the scope of what was publicly available."[30] In response, the prosecution claimed that the software was privileged as "a sensitive, non-public investigatory method."[31] Supported by an affidavit from TLO's founder, the prosecution argued that permitting the defense to test an executable copy would expose sensitive information stored on the developer's cloud server and would fail to solve the verification problem without additional disclosure of source code.[32] Meanwhile, TLO refused to disclose the source code even to law enforcement users, arguing that disclosing the code "to any law enforcement officer or agency at the municipal, state or federal level" would "compromise both current and future criminal cases where these tools are utilized."[33] Eventually, the *Clements* court was able to avoid ruling on the privilege claim when the parties agreed on a compromise whereby an independent expert would inspect the program and provide a response to a set of agreed-upon questions.[34] However, this compromise failed to solve the trust problem for either party, leaving the defense dependent on vague assertions from an expert it did not endorse and law enforcement dependent on the security precautions of a third party.[35] Ultimately, the defense was not satisfied with the expert's report, but the court refused further discovery and the case resolved in a plea bargain.[36]

[30] *Id.* at *5.

[31] Opposition to Defendant's Motion to Compel, U.S. v. Clements, 15-cr-275, ECF 18, at *6 (N.D. Ohio Jan. 26, 2016).

[32] Affidavit, U.S. v. Clements, 15-cr-275, ECF 18, at *5, 8 (N.D. Ohio Jan. 26, 2016).

[33] *Id.* at *8-1 ¶ 12.

[34] Minute Order, U.S. v. Clements, 15-cr-275 (Mar. 17, 2016).

[35] Motion, U.S. v. Clements, 15-cr-275, ECF 25, at *4-5 (N.D. Ohio April 18, 2016). This document may be found in this project's public repository on `github.com/BitanDor/NIT-ZKP at 25.0 Clements Expert Analysis Main Document.pdf`.

[36] U.S. v. Clements, 15-cr-275, ECF 34 (N.D. Ohio Feb. 6, 2017). We give more background on employing the independent expert in *Clements*. First, we note that the questions posed to the expert were phrased in English and included statements of professional opinion, which made them mathematically ambiguous. Namely, the expert was required to verify the following three assertions regarding the modified version:

Assertion 1. "The law enforcement software does not exploit a vulnerability in any software."

Assertion 2. "The law enforcement software does not misuse the protocols on the [P2P] network to do things that it was not intended to do."

Assertion 3. "The law enforcement software does not change any configuration options in other users' software to extract information that would normally not be publicly available (e.g. it does not make other users' software share private parts of their file systems)."

First, assertion 1 is not well-defined, since it is not clear which software the assertion considers for vulnerabilities. Second, assertion 2 is also not well-defined, because what constitutes "misuse" and what constitutes something that a software was "intended to do" could be litigated in different ways. Furthermore, any modification of a software arguably alters it to do something different from the original intent.

Ambiguity was not the only problem. To verify these assertions, the defense asked that the expert would also have access to an executable copy of the program, but the expert eventually received access only to the source code. The expert was still able to provide a report with a response to the agreed questions. That report supported the government's claims and concluded that the assertions were true. However, at that point, the defense counsel filed a motion for a second expert review, detailing the limitations of an expert review that lacked access to an executable version of the program.

As mentioned, in this article, we present a zero-knowledge prover-verifier protocol that enables a defense expert witness to verify a number of rigorously stated, precise assertions regarding the software, without law enforcement having to disclose the source code – or indeed an executable copy of the program. We argue that the proposed assertions, being precise, would suffice for verifying the prosecution's high-level claims, to the extent that these claims can be substantiated. Specifically, while assertions 1 and 2 are worded using subjective terms, our assertions unequivocally imply assertion 3.

1.2 Using ZKP to change the discourse

Initial goal. The above two cases demonstrate different ways in which the current system failed to obtain its stated truth-seeking goal. In *Budziak*, the two sides insisted on their perceived rights, but in the end the question of guilt remained unresolved and the validity of the software remained undetermined. In *Clements*, the two sides first appeared to have reached a compromise, only to realize that the compromise was ill-defined and insufficient. Again, the validity of the software remained in question.

Our goal in this work is to devise a system that uses Zero Knowledge technology to enable law enforcement to build (or procure) software for hidden P2P investigations such as the ones in *Budziak* and *Clements*, and then to provide convincing responses to legitimate challenges by the defense while preserving the secrecy and efficacy of the software. Specifically, our goal is to build a system such that, had this system been in use in *Budziak* or *Clements*, the discovery process would have resulted in a rigorous assertion of the validity of the software, while keeping it hidden.

Background on Zero Knowledge. ZKPs [24] are a mathematical concept that captures a class of algorithmic methods that enable two mutually distrustful parties, a *prover* and a *verifier*, to obtain the following joint goal. The prover holds two pieces of data: a secret piece X and a corresponding public piece D. Furthermore, X satisfies some mathematical property C. (In our context it will be convenient to think of the property C as a computer program that reads X and outputs ok if and only if the property holds. We henceforth refer to C as the *check program*.) The prover then discloses C and D and wishes to convince the verifier that she has data X which corresponds to D, and that satisfies property C — while at the same time making sure that X remains hidden from the verifier.

More specifically, a ZKP system consists of two public algorithms, ZK-Prove and ZK-Verify , that allow the prover and verify to perform the above interaction in spite of their mutual distrust. That is, the prover is expected to run ZK-Prove on input X, D, C, and the verifier is expected to run ZK-Verify on input D, C. The two algorithms exchange messages until such point where ZK-Verify outputs a decision. The verifier is then guaranteed that if ZK-Verify outputs ok then indeed the prover holds data X that corresponds to D and such that $C(X) = \text{ok}$. Furthermore, this holds even if the prover ran some other arbitrary program that's different than the prescribed ZK-Prove . At the same time, the prover is guaranteed that the verifier learned nothing about X, other than that X corresponds to D and that $C(X) = \text{ok}$. Overall,the effect of running the ZKP would be that of having an imaginary "genie" that gets D, C, X from the prover, and hands the verifier D, C along with a guarantee that it has seen data X such that X corresponds to D and $C(X) =$'ok'. See [3] for a primer on some algorithmic methods for obtaining ZKPs.

ZKPs have been extensively studied in the last four decades.[37] Recent years have brought a number of actual ZKP systems, mainly focused on identification and cryptocurrency applications.

Using ZKPs in legal settings was proposed by Kroll et al. [35, 36]. Specifically, they suggest using ZKPs to increase accountability for decision-making via secret algorithms, by providing a guarantee

[37] See, e.g., [4, 7, 8, 11, 12, 14, 16, 18, 19, 21–23, 26–28, 45].

that the algorithm performs in the manner described by policymakers. Bamberger et al. [3] define the concept of *verification dilemmas* as "costly choices between opportunities that require the verification of some fact, and risks of exposing sensitive information in order to perform verification", convey that the existing legal solutions to verification dilemmas are often imperfect, and envision using ZKPs to solve verification dilemmas throughout the legal landscape.

The proposed system: General workflow. We first present the general proposed workflow for using ZKPs for verifying properties of hidden investigative software. Next, we present our design and proof of concept implementation in more detail.

Commit. The process starts with the prover (namely, law enforcement) preparing the secret data X and public data D for the ZKP.[38] The secret data is the hidden software itself.[39] The public data D is then computed as "digest" of X that hides X and yet uniquely identifies X.[40]

Initial Check Program Design. Next, law enforcement determines the check program C. This has to be done judiciously, so that: (I) learning that $C(X) = \mathsf{ok}$ will indeed allay the concerns of the verifier (the defendant) regarding the operation of the software, and yet: (II) exposing the fact that $C(X) = \mathsf{ok}$ does not compromise the secrecy or efficacy of the software as an investigative tool. Law enforcement then hands D and C to the defendant.

Check Program Negotiation. The defendant (possibly aided by a domain expert witness) inspects the check program C and determines whether it suffices for allaying her concerns regarding the operation of the software. If not, the defense may request an alternate check program, and if the parties cannot reach agreement, adjudicate the scope the check program before the judge.[41]

Verification. Once the sides have agreed on a check program C, the parties execute the ZKP: law enforcement runs ZK-Prove with input X, D, C, whereas the defendant (or, the court) runs ZK-Verify with input D, C as described above, and verifies that ZK-Verify outputs ok .[42]

POC solution for P2P investigative software. We exemplify the efficacy of the above workflow for the case of verifying properties of investigative software of the style used in *Budziak* and *Clements*, namely software that runs on law enforcement devices and mimics the behavior of an actual P2P application, while requesting all pieces of a given file from a single target device. Specifically, we consider implementing check programs C that verify two main properties:

(a) The only information that the investigative software sends to the external network consists of legitimate data download requests that were generated via the original, public P2P application.

(b) The investigative software informs its operator that it has received a complete file F from IP address A only once it has successfully received all the pieces of file F, where each piece was received via the original, public P2P application, and all pieces came from IP address A.

Credibly performing such checks on an arbitrary piece of software is a daunting task, even if the software is presented in a convenient high-level language, and even without the need to later transform the check program to a ZKP. To make this task more feasible, we take the liberty to impose the following structural restriction on the software: We require that the software consist of two separate modules:

A *core module.* This module is arbitrary, except that it is required to not contain any network or I/O calls, or to communicate with any process other than the communication module.

A *communicator module.* This module is public and fixed, and functions as follows:

1. Given a request from the core to send a request to fetch piece i of file F from user U of the P2P network, the communicator runs the underlying P2P code to generate a P2P request to user U to obtain piece i of file F.
2. When receiving (from the TCP/IP port) a P2P incoming message with piece i of file F from user U, the communicator checks that the piece is received successfully by the P2P program and then records the receipt of that piece.
3. Once the list of file pieces received from user U contains all pieces of file F, the communicator informs the operator of the software of this fact.

We call this structural requirement on software *split architecture.* Now, all that the check program needs to verify is that the input software has split architecture, and that the communicator module is the one sketched above.[43]

Proof of Concept Implementation. To demonstrate the efficacy of our approach, we have built the following "toy system":

(I). We built a toy demo of a P2P file-sharing program, following the outline of the popular BitTorrent software [44], and set up a virtual network of users.

(II). We designed a modified version of the demo, that follows the general outline of law enforcement hidden investigative software for P2P networks as described above. Hereafter, we refer to the modified version as *a HIS* (Hidden Investigative Software). The HIS has split architecture as described above.

[38] Throughout, we consider law enforcement to be the prover. However, the exact same mechanism would allow an external vendor to prove properties of the software to law enforcement during procurement, where law enforcement plays the role of the verifier. Furthermore, some ZKPs are *transferable,* namely they allow law enforcement to later present the proof to a defendant or to the court and have them verify the validity of the vendor's claims without having to trust law enforcement in the process.

[39] In our proof of concept we only use the source code. However in some cases executables or other intermediary representations of the software might need to be included as well.

[40] That is, the prover sets $D = H(X)$ where H is a public algorithm such that: (I) D hides X, namely X remains secret even given $D = H(X)$, and (II) D uniquely determines X, namely it is not possible to come up with two different "openings" of the same D, or $X \neq X'$ s.t. $H(X) = H(X')$.

[41] While the possibility of a privilege dispute has not been eliminated, the judge now has far more fine-grained and nuanced options to order some disclosures rather than an all-or-nothing choice of ordering disclosure of the entire program. While in principle such nuanced disclosures could be made with the help of an expert witness, the ZKP allows doing this more securely, without any third-party seeing the code and without requiring the defense to take the third party at their word.

[42] We note that in our proof of concept implementation the interaction between the two algorithms consists only of a single message sent from ZK-Prove to ZK-Verify . In fact, this is a general property of many practical ZKPs. Still, this property is not essential for our workflow.

[43] Indeed, verifying that arbitrary software complies with any set of restrictions, even the relatively simple "sandboxing" ones listed above, is a daunting task. Still we argue that, in our setting, even a relatively superficial sequence of static checks, while not fool-proof, would provide a meaningful level of assurance. Indeed, subverting even a simple set of checks would require law enforcement developers to intentionally lie. Furthermore, such a lie would be recorded and subject to future exposure, say during audit.

(III). We designed a check program C as described above. Namely, the check program asserts that the core component does not contain programming commands essential for interacting with other network users and contains a call to the communicator (which is a proxy to the fact that it cannot interact with other users in any way except for using the communicator).
(IV). We implemented the check program in Cairo, which is a new ZKP-friendly programming language designed by StarkWare [7, 20]. Though Cairo supports general computations, it is designed mainly for programs that manipulate and validate properties of integers and is ill-suited for programs that manipulate text files.[44] To implement the check program in Cairo, we develop a variety of text-processing tools *in Cairo*. These tools enable writing Cairo programs that validate properties of strings and text files.
(V). We used StarkWare's *SHARP* (Shared Prover) software to generate a (single message) proof attesting to the integrity of the execution of our check program over the HIS source code.
(VI). Finally, we used StarkWare's *Verifier* software to verify the proof. (In terms of the above workflow, *SHARP* instantiates the abstract algorithm ZK-Prove and *Verifier* instantiates ZK-Verify .)
We note that the above POC is not complete, in that the StarkWare's *SHARP* is currently available only as a remote service, which means that one needs to send the hidden input X (along with D and C) to StarkWare's server. Furthermore, the proofs generated by the current version of *SHARP* are not known to be Zero Knowledge. Still, while both issues are serious show-stoppers in our setting, we argue that they are superficial and transient. In particular, SHARP is "almost Zero Knowlede": It is open-source and readily extendible to a version where the proofs are indeed Zero Knowledge. In addition, it can be built locally.

ZKPs for Text Processing. Our proof of concept implementation provides the following additional insight: Constucting ZKPs for text processing problems is significantly easier than constructing multi-party computation (MPC) protocols for the same. Indeed, text processing is known to be notoriously difficult to cryptographic primitives such as fully-homomorphic encryption (FHE) schemes, and secure multiparty computation (MPC) protocols as FHE and MPC are based largely on translating programs to circuits whose basic operations are algebraic operations on numerical representations of the input, which essentially require evaluations of degree d polynomials over finite fields, where d corresponds to the depth of the circuit. Since in the case of text processing, neither the input nor the operations are natively arithmetic but rather operate with Boolean inputs and operations, text processing in an FHE or MPC setting often (but not always) translates to high degree polynomials and prohibitively slow operation. In contrast, our work suggests that applying ZK proof to prove a property of text or that a correct run of a text-processing task has been executed, can be done efficiently circumventing the algebraic vs. Boolean issue. This is due to the fact that, in ZKPs, the prover can deal with the input and program instructions in the clear, whereas secrecy requirements only come into play for the verifier. We illustrate this by adopting Cairo's algebraic structure to support the secure verification of an execution of a text-processing task.

Cairo Bag-of-words. Benefitting from the above insight, we enhanced our Cairo POC to develop Cairo tools for *bag-of-words* (BOW). BOW is a well-studied and known-to-be-useful text-processing technique in the field of NLP (natural language processing). Given a text file, its BOW is a list that indicates the number of appearances of each word that appears in the text. Computing the BOW of a text file is extremely useful in many natural language processing (NLP) applications [25, 50, 57]. The Cairo-BOW programs we developed in this work may be used in various settings, perhaps NLP-related or other. These tools are presented in the Appendix.

1.3 Implications for Law and Legal Theory

Our proposal improves over the existing baseline of evidentiary privilege law for both good faith and bad faith scenarios. Our proposal enables good faith law enforcement to corroborate that hidden investigative software is constitutional, lawful, and accurate without disclosing an executable copy of the software or its source code, hence maximizing transparency and disclosure while protecting legitimate secrecy interests and simultaneously bringing a case to trial. Our proposal also enables good faith defense counsel to verify the same attributes, and thereby to vindicate the client's Fourth Amendment and fair trial rights. Meanwhile, our proposal makes it harder for bad faith or negligent law enforcement to conceal wrongdoing or inaccuracy because, if ZKPs become standard, it will be harder to submit a fraudulent or mistaken affidavit that inaccurately describes how the software works without getting caught. Moreover, even pretextual or overbroad claims to the law enforcement privilege will no longer result in withholding substantial information about a hidden investigative software. Lastly, our system makes it harder for bad faith defense counsel to graymail law enforcement into dropping justified charges when no wrongful government conduct has occurred since filing a frivolous discovery motion with the motive to compel dismissal will no longer be an effective litigation strategy. See Subsection 4.1 for more discussion.

From a more conceptual perspective, we observe a mode of interaction between CS research and law. We propose that demonstrating a POC of technological feasibility can support novel legal arguments even before adoption of a new technology. Specifically, feasibility alone may lead some courts to decide that non-adoption is unreasonable, and apply existing legal balancing tests differently as a result. Hence, common law courts can encourage adoption of new technological improvements where statutory or regulatory interventions would be limited. See Subsection 4.2 for further discussion.

1.4 Paper outline.

The rest of the paper is organized as follows. Section 2 provides further legal background on the hidden investigative software problem. Section 3 presents our POC solution. The legal implications of our system are discussed in Section 4. Finally, the possibility of using our tools and methods for solving other verification dilemmas is discussed in the Appendix, along with supplementary material.

[44] In fact, Cairo does not support integers directly, but rather it supports *felt* variables. Felts are "field elements" – integers modulo some large prime number. We note that the Cairo compiler *can* process short-strings, which can be as long as 31 characters. However, a short-string is simply a way to represent a field element, where the string's first character is the most significant byte of the integer (big-endian representation). This is not a real string.

2 THE LAW ENFORCEMENT SECRECY PROBLEM

2.1 The Law Enforcement Evidentiary Privilege

The tension between law enforcement secrecy and fair trial rights in criminal cases has long plagued the courts. Two famous historical cases illustrate opposing resolutions of this conflict. The first is the eighteenth-century English trial of Bishop Atterbury for treason. In investigating this case, the government seized an encrypted letter from the mail. The prosecution then proffered an expert witness to testify at trial that he had successfully decrypted the letter, and that the decrypted contents established Atterbury's guilt. But when the defense sought to challenge the accuracy of the decryption process, the prosecution refused to disclose any information about "the method and manner of decyphering"[45] on the basis that doing so would "instruct ill-designing men to contrive more difficult cyphers."[46] In other words, the prosecution argued that it should be entitled to introduce the results of the decryption process as evidence of guilt without subjecting the decryption process to defense scrutiny. The defense complained that, in this circumstance, "our lives and fortunes must depend on the skill and honesty of decypherers . . . when there are not means of contradicting them[.]"[47] Nonetheless, the English court ruled for the prosecution. It permitted the witness's testimony but not the defense cross-examination, reasoning "that it is not consistent with the public safety, to ask the decypherers any questions, which may tend to discover the art or mystery of decyphering."[48] In sum, even at the guilt phase of trial, the court prioritized protecting the efficacy of law enforcement investigative methods over the defendant's ability to confront and cross-examine a key witness against him.

The second case is a twentieth-century opinion by Judge Learned Hand writing for the U.S. Court of Appeals for the Second Circuit. In *U.S. v. Coplon*,[49] a defendant was charged with espionage for the Soviet Union in the midst of the cold war. She alleged that the FBI had used illegal wiretapping to initiate the investigation against her. If correct, this fact would have enabled the defense to suppress evidence of guilt derived from the tainted investigation, potentially crippling the government's case. In an attempt to disprove the allegation, the prosecution submitted confidential FBI files to the judge for *in camera* review but refused to allow defense counsel to examine them. Judge Hand acknowledged that the government has a privilege to withhold certain sensitive information from the defense, such as "the names or statements of informers."[50] But he ruled that the prosecution could not assert such a privilege to conceal information from the defense while simultaneously seeking to rely on the very same privileged information in support of its case.[51] Even for a pre-trial suppression motion, the court held that if the government chooses to rely on privileged information, then the privilege must yield to a criminal defendant's right to challenge the government's case. In Judge Hand's memorable words, "Few weapons in the arsenal of freedom are more useful than the power to compel a government to disclose the evidence on which it seeks to forfeit the liberty of its citizens. . . . A society which has come to wince at such exposure of the methods by which it seeks to impose its will upon its members, has already lost the feel of freedom and is on the path towards absolutism."[52]

It is against the historical backdrop of cases like Bishop Atterbury's trial and *Coplon* that twenty-first-century prosecutors are asserting a qualified law enforcement evidentiary privilege to withhold methodological details about hidden investigative software from criminal defense scrutiny. In most of these cases, law enforcement has relied on results generated through the use of hidden investigative software to obtain warrants to arrest people or seize property. In those cases, defendants have sought information about how the software works to verify its correctness, and furthermore that its use was lawful and constitutional – else, as in *Coplon*, evidence derived from the software might be tainted and require suppression. For instance, if the software accessed private folders or altered data or settings on a target device, those actions could violate the Fourth Amendment as well as cast doubt on the accuracy of the results. In a smaller set of cases, law enforcement has also sought to rely on the results generated through the use of the software as direct evidence of guilt. In those cases, as in Bishop Atterbury's trial, the defense has sought information about how the software works to contest the accuracy of the techniques and the credibility of the prosecution witnesses who testify about their results. In either circumstance, the modern law enforcement evidentiary privilege doctrine controls the resolution of these conflicts.

2.2 The Alternate-Availability Factor

Many courts analyzing law enforcement privilege claims have applied a balancing test drawn from a federal district court case within the U.S. Court of Appeals for the Third Circuit, *Frankenhauser v. Rizzo*.[53] *Frankenhauser* lists a series of factors for courts to consider when weighing law enforcement's interest in secrecy against the defendant's interest in disclosure. The most pertinent factor for our purposes is "whether the information sought is available through other discovery or from other sources."[54] This factor can cut both ways. On the one hand, if the precise information sought is already available elsewhere, then it is less likely that disclosing that information to the defendant under a protective order would harm law enforcement interests, and hence law enforcement should have less reason for secrecy. On the other hand, if the defense can obtain the same information elsewhere, then the defendant should have less need to compel its disclosure from law enforcement.

For hidden investigative software, most courts to date have sided with law enforcement and upheld claims to the privilege.[55] Although the judicial reasoning in these cases is consistently thin,

[45] 1723 Bishop Atterbury's Trial, 16 How. St. Tr. 497.
[46] *Id.*
[47] *Id.* at 672.
[48] *Id.* at 497.
[49] U.S. v. Coplon, 185 F.2d 629 (2d Cir. 1950).
[50] *Id.* at 638.
[51] *Id.* (reasoning that it is "one thing to allow the privileged person to suppress the evidence, and, toto coelo, another thing to allow him to fill a gap in his own evidence by recourse to what he suppresses").
[52] *Id.*
[53] Frankenhauser v. Rizzo, 59 F.R.D. 339 (E.D. Pa. 1973).
[54] *Id.* at 344
[55] *See, e.g.*, U.S. v. Pirosko, 787 F.3d 358 (6th Cir. 2015). One federal court characterized *Pirosko* as spawning a "line of cases" that deny criminal defense teams "access to confidential government investigative software." U.S. v. Gonzales, 2019 WL 669813, at *3 (D. Ariz. Feb. 19, 2019).

one explanation for their outcomes is the alternate-availability factor that *Frankenhauser* identifies, namely that judges view affidavits from law enforcement officers attesting to how the software works as providing a sufficient alternative source of information for the defense so as to remove the defendant's need to access source code or specific deployment details.[56] Still, other judges disagree with such views.[57]

Adding ZKPs to law enforcement's use of hidden investigative software would interact with the current evidence law doctrine primarily through this alternate-availability factor. For instance, if a ZKP can convey that a software does not access private folders or alter data or settings on a target device, then the ZKP would offer better alternative evidence for the defense than an affidavit from a law enforcement officer or software developer alone. An affidavit requires the defense to trust the witness's testimony about how a software works, even while the privilege denies the defense access to information that might impeach that testimony. A ZKP, in contrast, would add further assurances that a witness's description of how the software works is neither willfully false nor mistaken. Furthermore, ZKPs enable a defense domain expert to directly investigate the software in a precise and fine-grained way. Such investigation can be significantly more meaningful than cross-examining a human intermediary whose knowledge of the software may be incomplete.

3 THE PROPOSED SYSTEM AND POC

This section presents our solution to an instance of the hidden investigative software problem and an implementation that follows the solution.

3.1 The solution at high level

The target instance. In the target instance, law enforcement investigators use a modified version of a P2P software to investigate the activity of CSAM distributors. Hereafter, we refer to the modified version as 'the HIS' (for hidden investigative software). To vindicate the defendant's right to determine the accuracy, lawfulness, and constitutionality of the tools used to investigate the case, the defense expert wishes to verify that all messages sent from the HIS to the target user are identical to requests that could have been sent from any other peer in the network that uses the non-modified version. However, as mentioned, law enforcement wishes to keep the HIS hidden to maintain its future efficacy. This is the target instance.

Our solution in high-level. Our solution for the target instance adopts the four-steps workflow mentioned in Section 1: (a) commitment; (b) check program initial design; (c) check program negotiation; (d) verification. In terms of this workflow, the secret data X is the HIS, and the check program C is the process that inspects the HIS and verifies that the specified properties hold.

Rather than designing a check program that checks the entire HIS, we instead impose the following simple structural restriction on the HIS – it should be designed according to *the split architecture* principle (also illustrated in Figure 1). The split architecture obliges the HIS to be made of two separate components – *the core component* and *the Communicator*. The core component is the main component of the program – it handles all its local processes (e.g., graphical user interface, local memory read/write, and all local computations). The Communicator is used for Internet communication, and particularly, to send messages to other network users. The core component does not contain any pieces of code that can interact with the Internet.

This architecture allows us to keep the check program C simple and practically realizable. In our solution, the Communicator generates, sends, and logs messages identical to messages sent by the non-modified version of the P2P software. Hence, in our solution, the Communicator is published with no risk of exposing secret information enclosed in the core component. Publishing the Communicator, anyone can check that the messages it generates are indeed legitimate, in the sense that they could have been generated by users of the non-modified version of the P2P software. Further more, the Communicator logs all the interactions of the program, and generates a Download Summary Report with every download. Now, all the check program C has to verify is the following two attributes. First, the core component indeed calls the Communicator, and second, other than calling the Communicator, the core component contains no pieces of code that can interact with the Internet.

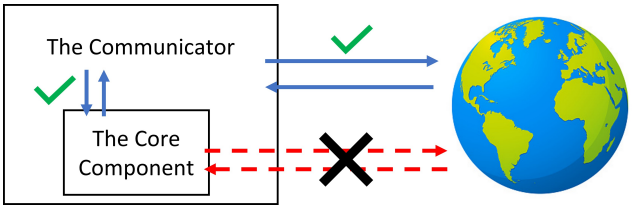

Figure 1: The split architecture. The core component interacts with the Internet via the Communicator. Blue arrows indicate the allowed interactions between the core component and the Communicator, and between the Communicator and the Internet. Red (dashed) arrows indicate that the core component cannot interact with the Internet independently.

We suggest and demonstrate how to design the check program by specifying two lists of strings – *blocklist* and *allowlist*. The check program verifies that no blocklist elements appear in the source code of the core component, and all allowlist elements do appear. Each string in the blocklist is either a keyword, a library name, a parameter, or a programming command required for writing programs that enable connecting to the Internet in the language the HIS is developed. The strings in the allowlist are commands by which the core component calls the Communicator.

The final step in our solution is implementing the check program in a programming language that natively supports generating ZK-proofs, hereby taking care of the verification step of our four-step

[56] *See, e.g.*, Opinion and Order, U.S. v. Pirosko, 12-cr-327, ECF 33, at *5 (N.D. Ohio Aug. 13, 2013) ("Defendant will still be able to have a fair trial without access to [the HIS]" because the government provided an affidavit and computer logs showing that the HIS "are not capable of manipulating target computers.").

[57] *See, e.g.*, U.S. v. Budziak, 697 F.3d 1105, 1112 (9th Cir. 2012); U.S. v. Liebert, 519 F.2d 542, 547–48 (3d Cir. 1975) ("A party seeking to impeach the reliability of computer evidence should have sufficient opportunity to ascertain by pretrial discovery whether both the machine and those who supply it with data input and information have performed their tasks accurately."); U.S. v. Dioguardi, 428 F.2d 1033, 1038 (2d Cir. 1970) ("It is quite incomprehensible that the prosecution should tender a witness to state the results of a computer's operations without having the program available for defense scrutiny and use on cross-examination if desired.").

workflow. The ZKP-friendly language should also support computing a hash function as mentioned in the commit step of the workflow.

3.2 The implementation

The BitTorrent demo. To lay the grounds for the implementation of our solution to the target instance, we first design a toy demo of a generic P2P software. For simplicity, we modeled our demo after BitTorrent, which is a contemporary and popular P2P system. Indeed, BitTorrent, developed by Bram Cohen in 2001 [44], is one of the most popular P2P programs in use today [56].

We begin with a high-level description of how BitTorrent works. To insert a file into the network and share it with other users, a BitTorrent user follows the following steps. First, the file is divided into small pieces, and the cryptographic hash of each piece is computed. These hashes are then used to build a hash tree, whose root serves as a unique identifier for that file. Next, a *torrent* file is built, which contains relevant information regarding that file, including its suggested name, size, number of pieces, piece length, the hash of each piece, and root of the corresponding hash tree. The torrent file also includes a URL address of a *tracker* for that file. A tracker is a website that indexes information regarding files in the network, including IP addresses of users that hold pieces of that file. Inserting a new file into the network is also called *seeding*, and the user first uploading the file is its *seeder*.

To download a file, a BitTorrent user searches a tracker for the file's torrent and the list of users holding its pieces. These users are identified by an IP address and a port number. Then, the user sends requests for pieces of that file to users who (according to the tracker) hold these pieces. After receiving the pieces from different users, the user computes the hash of each piece and the root of the corresponding hash tree. If these match the torrent data, the pieces are reassembled to the wanted file (and the user updates the tracker about possession of the downloaded pieces). We note that the BitTorrent software includes many additional components, some of which are designed to compel or forbid particular user behaviors, such as downloading without uploading. However, these aspects were not implemented in our proof-of-concept demonstration, nor would they necessarily be implemented in a law-enforcement HIS.

The HIS. Recall that, as discussed in Section 2, law enforcement officers investigating a P2P user for CSAM distribution aim to perform a single-source download, where all the pieces of a CSAM file are received from the same user. Hence, we program a modified version of our BitTorrent demo that does exactly that. That HIS works in the following way. Instead of sending pieces requests to different network users, the HIS locates a user that (according to the tracker) holds all the pieces of the requested file, and sends all the requests to that user. The requests are sent in the exact same way as the non-modified version requests are sent. In fact, the HIS uses the same component to send the requests as the non-modified version does. Using that HIS, a law enforcement investigator can perform a single-source download from a CSAM distributor.[58]

Designing the check program. After establishing the setting, we turn to design the check program, which is the main interest of steps (b) and (c) of our four-step workflow. To this end, it is helpful to use a programming language that has a "backend" that natively supports proof generation and verification.

We implement our check program in Cairo. Cairo is a new programming language designed by StarkWare, and it supports general computations and ZK proof-generation and verification. In [7], Ben-Sasson et al. present algorithms for generating succinct, transparent, and zero-knowledge proofs that attest to the integrity of the execution of Cairo programs over secret inputs. One of Cairo's advantages is enabling the generation of ZK-proofs doubly-efficiently, which means that both proof and verification times scale.[59] It also has a decent and growing library of gadgets that become handy in our setting. In particular, Cairo has a built-in that supports the computation of a cryptographic hash function (The Pedersen Hash function), which is essential for steps (a) and (d) of our four-step workflow.

Aside from its advantages, Cairo has several downsides. Presently, Cairo users cannot generate and verify proofs locally. Instead, StarkWare provides the *SHARP* (Shared Prover) and *Verifier* online services for proof generation and verification. Unfortunately, not only that these online services require users to disclose their program and its (perhaps secret) input to StarkWare, but also the current implementation of SHARP does not follow the exact steps specified in [7] for generating ZK-proofs. Hence, SHARP-generated proofs fall short of achieving the zero-knowledge requirement.[60] Besides that, Cairo is designed mainly for programs that manipulate and validate properties of integers and is ill-suited for programs that manipulate strings, floating-point numbers, and other commonly used variable types.[61] It can directly handle two types of variables: memory addresses, and field elements (*felts*) – integers modulo a large prime number.[62] In addition, Cairo has an immutable memory, does not support loops, and has other uncommon features. Though these features are essential for creating efficient ZKPs, they can make routine programming tasks non-trivial to carry. These are some of the reasons why writing Cairo programs that enable processing and analyzing text files can be a challenging task. We now describe how we handled this task and present the tools we developed to allow Cairo text processing.

The initial processing. The first tool we developed (in Python) is *the Initializer*. The Initializer gets a text file as input and carries two tasks – converting the input to a Cairo-readable format and implementing a cryptographic commitment. In more detail, as mentioned above, the secret digital object held by the prover is a text file (in our case, this text file is the source code for the core component of the HIS). In our four-step workflow, the text file is fed into a Cairo check program. However, as mentioned, Cairo does not support text variables. To overcome this obstacle, the Initializer converts

[58] The code for the HIS can be found in our project's public repository under `BitTorrent_Demo_LE_version.py`.

[59] Cairo achieves that based on a novel mathematical tool called FRI. FRI stands for Fast RS IOPP, which means Fast Reed-Solomon Interactive Oracle Proof of Proximity.

[60] We note that StarkWare's proof generation process only slightly deviates from the [7]-algorithm, and it is unknown if there exists any feasible way to take advantage of these deviations and extract secret information from the proof.

[61] In fact, Cairo does not support integers directly, but instead, it supports *felt* variables. Felts are "field elements" – integers modulo some large prime number. We note that the Cairo compiler *can* process short-strings, which can be as long as 31 characters. However, a short-string is simply a way to represent a field element, where the string's first character is the most significant byte of the integer (big-endian representation). This is not an actual string.

[62] We use Cairo version 0.7.0, where that number is $2^{251} + 17 \cdot 2^{192} + 1$.

the text file into an array of felts – a form ready to be processed by Cairo.[63]

Second, at the commit step of our four-step workflow, the prover computes a particular "digest" of the text file. The Initializer implements this stage and outputs such a "digest" value with the required properties listed above. In detail, the Initializer randomly picks a felt and appends it to the list of felts computed at the first stage. The enlarged set of felts with the randomness added is denoted X. Then, to achieve a semantically secure commitment of the text file, the Initializer applies tree-hashing to X using the Pedersen hash function (as implemented by StarkWare's `crypto` library for Python) and obtains a single felt D. Finally, the Initializer outputs X and D.

Below, whenever we say that a Cairo program gets a text file or a string, it gets an array of felts obtained from the Initializer.[64]

The check program. Once the text file is converted into an array of felts, it can be fed to Cairo programs. To implement the check program, we develop Cairo programs that get arrays of felts (as those output from the Initializer) and realize a variety of text-processing queries. These queries appear below.

3.2.1 Our Cairo text-processing queries. Cairo implementation of the following text-processing queries may be found in our project's public repository.[65]

- Does the text begin with a?
- Does the text contain a at least once?
- How many times does a appear in the text?
- If the text contains a, what is the word that appears right after the first appearance of a?
- If the text contains a, what are all the words that appear in the text right after a?
- What appears in the text before a?
- Given two strings a and b that appear in the text, what lies between these strings in the first time they appear?
- What are all the things that lie between a and b whenever they appear?

These queries are implemented as follows.

`X_start_with_arr`. This function gets a text file X and a string *arr* and checks whether X begins with *arr*. It returns 1 if it does, and 0 otherwise. Throughout the following, X and *arr* are arrays of felts, where X is longer than *arr*.

`X_contains_arr`. This function gets a text file X and a string *arr* and checks if *arr* appears in X at least once. It returns 1 if it does, and 0 otherwise.

`Count_arr_in_x`. This function gets a text file X and a string *arr*, checks how many times *arr* appears in X, and returns the number of appearances.

`Read_word_in_x_after_arr`. This function gets a text file X and a string *arr*. It returns the word that appears right after the first appearance of *arr* in X. If *arr* does not appear in X it returns 0.

`Read_all_words_after_arrs`. This function gets a text file X and a string *arr*. It returns an array that contains all the words in X that appear right after an appearance of *arr*. If *arr* does not appear in X it returns an empty array.

`Read_to_point`. This function gets a text file X and a string *arr*. It returns an array that contains all the words in X that appear before the first appearance of *arr* in X. If *arr* does not appear in X the function returns the content of X, except for its last l entries, where l is the length of *arr*.

`Read_between`. This function gets a text file X and two string – a and b. The function returns the first substring of X that lies between a and b (including a and excluding b). If a does not appear in X, the function returns an empty array. If a does appear in X but b doesn't, the function returns the part of X that starts with the first appearance of a, up until the end of X, except for its last l entries, where l is the length of b.

`Get_all_between`. This function gets a text file X and two string – a and b. The function returns all the substrings of X that lie between a and b. If a does not appear in X, the function returns an empty array. If a does appear in X but b doesn't, the function returns the parts of X that starts with an appearance of a, up until the end of X, except for its last l entries, where l is the length of b.

We use these programs to design a Cairo check program C. We note that in our solution, D is hard-coded in C. Our check program gets the source code for the core component of the HIS as input and does the following. First, C applies the Pedersen hash function to its input the same way the Initializer applies it to the enlarged list of felts. C verifies that the outcome matches the hard-coded D. Second, C performs a list of hard-coded text-queries and assertions on the input as specified by two lists – *block* and *allow*. The check program verifies that no blocklist elements appear on the source code of the core component, and all allowlist elements do appear. The blocklist contains keywords, library names, parameters, and programming commands essential for writing Python programs that enable connecting to the Internet. In our case, we generate the blocklist based on the syntax, semantics, and standard libraries of Python.[66] The allowlist contains the actual commands that appear in the core component of our HIS for calling the Communicator.

[63] Each felt in that array represents a single word. Since our primary goal is tools that offer evidence regarding attributes of computer programs, the partition of the text into words and the conversion to felts are implemented in a way that brings the source code into a canonical form while maintaining the structure and the compile-ability of the program. Namely, the Initializer encodes indentations and end-of-lines in a special way, ignores redundant spaces within the line and redundant blank lines (as the compiler does), and considers certain single-characters as words (e.g., periods, parentheses, colons, semicolons, mathematical operations, and other signs with special meaning in programming). We chose this particular way of converting text into Cairo felts to meet our goals. This design can readily be modified if the text files processed are of a different type (which requires keeping track of redundant spaces within the line, blank lines, etc.).

[64] The Initializer source code appears in our project's public repository at `process_input_for_cairo.py`.

[65] At `text_query_tools.cairo`.

[66] If the P2P software is coded in a different programming language, one may design a different blocklist accordingly.

If one of the assertions fails, the check program crashes, and no output is delivered.[67] The check program's steps appear below.[68]

3.2.2 Main steps of the Check Program.

(1) The Check Program computes the Pedersen hash of its inputs and asserts that it matches the hard-coded D.
(2) `appear_at_least_once` is a list. For each string s in that list, the Check Program calls `x_contains_arr` to assert that s appears in X at least once. Particularly, this list contains commands that show that the HIS calls the Communicator.
(3) `do_not_appear` is a list. For each string s in that list, the Check Program calls `x_contains_arr` to assert that s does not appear in X. This list contains programming commands and other parameters essential for communication with network users.
(4) `appear_exact` is a list. Each element is a pair (s, c), where s is a string and c is an integer. For each pair, the Check Program calls `count_arr_in_x` and asserts that the number of times that s appears in X is c.

We note that, for convenience, we developed a Python program that gets block and allow lists and a digest D and generates a Cairo check program.[69]

3.3 The execution

The desired execution. Ideally, we would like to use the tools and methods suggested above to realize the four-step workflow and implement our solution for the target instance of the HIS problem.
Commit. The prover executes the Initializer over the source code of the core component of the HIS to obtain a list X of Cairo felts and a digest D. The prover publishes D. This step can be done well in advance, long before the need to carry the next steps has emerged.
Check program. The prover and verifier follow steps (b) and (c) of our four-step workflow to obtain two lists (block and allow) as mentioned above and build a Cairo check program accordingly (with D hard-coded).
Prove. The prover runs C over X, and follows the [7]-algorithm[70] for proof generation to generate a ZK-proof P attesting to the integrity of the execution of C over X. The prover sends P to the verifier.
Verify. The verifier follows the [7] verification algorithm to verify P.
The actual execution. We use the tools described above to execute the following steps.[71] We execute the Initializer over the core component of our HIS to obtain X and D. Next, we design block and allow lists, and use them (and D) to code a Cairo check program C, and run it over X. In these steps, we use Cairo's SHARP services to generate a (non-ZK) proof attesting to the integrity of the execution of C over X [49]. Submitting the job to SHARP, we obtain a *fact* and a *job key*. We use these values to employ StarkWare's Verifier software to verify the SHARP-generated proof [48]. Since we use StarkWare's services for proof generation and verification, our implementation does not fully realize the solution suggested here. However, we believe that as far as our proof-of-concept implementation is concerned, this lacuna is transient. This gap may be resolved in future work.

4 LEGAL IMPLICATIONS ANALYSIS

4.1 Improving Over the Legal Baseline

To implement our POC, law enforcement should adopt the following two steps. *First*, when a law enforcement agency acquires a hidden investigative software, it should compute a cryptographic hash of the software source code and either publish the hash publicly or, if doing so would create a security risk, store the hash with a trusted third party. *Second*, if the software functionality later becomes relevant in litigation, then law enforcement can disclose the original hash to the defense and run a ZKP on the software (as suggested above in Section 3). Importantly, in the scenario where law enforcement procures a software from a private developer that refuses to reveal the source code or other pertinent details, our same protocol should apply. Law enforcement should play the role of the verifier and the developer should play the role of the prover. This improves over the legal baseline for the following reason. Currently, either the defense must trust a law enforcement affidavit or law enforcement must trust a defense expert not to leak. Using our system provides a third option in which the defense can verify that the hidden investigative software operates in a legitimate way without inspecting the source code or an executable copy, while law enforcement can eliminate the risk of leaks.

To be sure, our proposal is not foolproof. Bad faith law enforcement could still cheat in at least two ways. *First*, they might conceal the use of an unlawful software by initially hashing and then later running the ZKP on a dummy software, rather than on the software that was actually used to investigate the defendant. But the same risk is already present without our solution. For instance, if a court orders source code disclosed to a defense expert witness, law enforcement could willfully or mistakenly disclose the wrong code. Indeed, human rights organizations have documented a recurring phenomenon, called "parallel construction," in which law enforcement conceals potentially unlawful or unconstitutional investigative conduct from the courts by undertaking a subsequent, lawful investigation to describe in court filings [52].

While adding the hash plus ZKP protocol would not entirely eliminate these risks, it would reduce them. To run the ZKP on a dummy code, a bad faith law enforcement officer would have to obtain and hash the dummy code well in advance of litigation, prior to or perhaps contemporaneous with deploying the true software in the investigation, and perhaps before a motive to fabricate has arisen. Meanwhile, this advanced planning would make it harder for an individual law enforcement officer to conceal false testimony or disclosures, harder for the officer to claim that any erroneous disclosures resulted from innocent mistake, and harder for the officer to make innocent mistakes.

Second, bad faith law enforcement could theoretically design an obfuscated code with attributes that would be hard to identify by text-processing queries like those suggested in our solution. But,

[67] We note that it is impossible to generate a proof regarding a crashed execution of a Cairo program.
[68] The source code for the check program may be found at our public repository in `Wrapper4paper.cairo`.
[69] This program may be found in our project's public repository under `Gen_W_for_Cairo.py`.
[70] See Theorems 1-2 in pages 8-9, a standalone construction on pages 23-27, and further details in [6].
[71] In our proof-of-concept demonstration, both the prover and verifier steps were carried out on the same machine (a home-use laptop).

again, similar risks are already present without our solution. For instance, a law enforcement officer could mis-describe the software function in an affidavit, whether willfully or by mistake.

Once again, while adding the hash plus ZKP protocol would not entirely eliminate these risks, it would reduce them. Not only would designing an obfuscated version require advanced planning and establish deliberate malfeasance that raises the stakes of getting caught, but also it would require substantially more effort than after-the-fact lying in an affidavit. Of course, law enforcement could also disclose obfuscated code to a defense expert witness. In this scenario, defense expert review may be preferable to our solution. Regardless, we assume that obfuscation is a low risk scenario due to the high costs to law enforcement of doing this. In short, the options for cheating our ZKP protocol already exist in the current legal baseline, and our ZKP proposal would make them harder to accomplish.

4.2 Conceptual Contributions to Law

Legal scholars have richly debated how technology and law interact [2, 13, 15, 40].[72] We observe a mode of interaction between computer science *research* and law: demonstrations of technological feasibility, even before general adoption of a new technology, may immediately enable new legal arguments.[73]

There are good reasons to think that law enforcement will resist voluntarily adopting ZKPs for hidden investigative software. Implementing ZKPs would require substantial advanced planning, as well as resources and expertise that many law enforcement agencies may lack. Further, the no-ZKP status quo benefits law enforcement. In most recent cases, courts have upheld claims to the law enforcement privilege and permitted prosecutions to proceed on the basis of mere affidavits alone, without requiring additional disclosures about hidden investigative software that could enable the defense to test or impeach law enforcement claims. While law enforcement should ideally want to maximize transparency to the defense and subject itself to adversarial scrutiny, repeated complaints about an "epidemic of *Brady* violations" suggests that reality can be otherwise [1].

Presuming that our ZKP proposal best serves the public interest, how can law encourage its adoption?[74] We propose that a novel interpretation of existing evidentiary privilege law can help solve this conundrum. The current *Frankenhauser* alternate-availability factor is retrospective, meaning courts consider whether an alternative, non-privileged source for the pertinent information already exists. But this factor could also be prospective, such that courts would consider the theoretical feasibility of providing an alternative, non-privileged source for the information.[75] Accordingly, knowing that ZKP solutions are feasible, courts could require them, either by denying privilege claims and ordering disclosure where law enforcement has failed to adopt a ZKP mechanism or by ordering use of ZKPs as a form of prospective relief.[76] As ZKPs become more commonplace throughout society, law enforcement failure to adopt them to facilitate enhanced disclosures to the defense may begin to seem unreasonable, and courts may be more willing to consider their feasibility in the *Frankenhauser* analysis. Our POC is a step in the direction of making ZKPs the best practices standard for designing law enforcement investigative software.

ACKNOWLEDGMENTS

We want to thank Eli Ben-Sasson for valuable comments and discussions, particularly suggesting using Starkware's Cairo programming language as a building block in the solution.

REFERENCES

[1] Appeal, T. The epidemic of brady violations: Explained. Accessed: 2022-02-01. URL: www.theappeal.org/the-epidemic-of-brady-violations-explained-94a38ad3c800/.
[2] Balkin, J. M. The path of robotics law. *Calif. L. Rev. Circuit 6* (2015), 45.
[3] Bamberger, K. A., Canetti, R., Goldwasser, S., Wexler, R., and Zimmerman, E. J. Verification dilemmas, law, and the promise of zero-knowledge proofs. *Law, and the Promise of Zero-Knowledge Proofs (February 7, 2021)* (2021).
[4] Barak, B. How to go beyond the black-box simulation barrier. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science* (2001), IEEE, pp. 106–115.
[5] Bell, C. M. Surveillance technology and graymail in domestic criminal prosecutions. *Geo. JL & Pub. Pol'y 16* (2018), 537.
[6] Ben-Sasson, E., Bentov, I., Horesh, Y., and Riabzev, M. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch. 2018* (2018), 46.
[7] Ben-Sasson, E., Bentov, I., Horesh, Y., and Riabzev, M. Scalable zero knowledge with no trusted setup. In *Annual international cryptology conference* (2019), Springer, pp. 701–732.
[8] Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., and Ward, N. P. Aurora: Transparent succinct arguments for r1cs. In *Annual international conference on the theory and applications of cryptographic techniques* (2019), Springer, pp. 103–128.

[72] For one example among many, Rebecca Crootof and BJ Ard argue that technological developments foster three recurring categories of legal uncertainty: how existing law applies in the new socio-technical context; whether existing law accomplishes its normative objectives in the new socio-technical context; and whether existing legal institutions suffice to resolve these questions[15].

[73] Relatedly, widespread adoption of ZKPs for hidden investigative software would change the facts to which judges apply current evidentiary privilege doctrine, but would be unlikely to motivate or require any change to the doctrine itself. Specifically, widespread adoption would strengthen law enforcement's arguments against disclosing source code or specific deployment details as unnecessary to the defense while simultaneously strengthening defendants' arguments for accessing more information than mere affidavits alone. As a result, most defendants would likely get access to more information than they receive today, without jeopardizing any legitimate law enforcement secrets, while the few defendants who currently obtain disclose-or-dismiss orders would be less likely to do so moving forward. Yet, to follow Crootof and Ard's framework for assessing the relationship between technological change and law [15], these changes would be unlikely to create application, normative, or institutional legal uncertainties. Privilege law would still clearly apply. And the existing *Frankenhauser* alternate-availability factor is easily capacious enough to accommodate the change in facts without modification of the existing legal rules. Moreover, the likely change in outcomes would serve the normative goals of both fair trial rights and the law enforcement privilege by enabling defense counsel to verify characteristics of hidden investigative software without jeopardizing legitimate law enforcement secrets.

[74] Regulatory approaches will face difficulties because law enforcement is highly decentralized. As of 2016, there were over fifteen thousand distinct law enforcement agencies in the United States [9, 31]. Meanwhile, federal authority to regulate state and local police is limited [17]. Constitutional criminal procedure approaches are also unlikely to succeed because, if most defendants cannot show sufficient need for information (beyond an affidavit) to win the balancing test in the qualified law enforcement privilege analysis, then ipso facto they cannot show the even greater constitutional need required to defeat the privilege [33]. And expert evidence admissibility rules, such as *Daubert* and *Frye*, offer minimal help for regulating investigative technologies that law enforcement primarily uses to generate probable cause for searches and seizures, rather than evidence of guilt for trial. *Daubert* and *Frye* are standards for controlling the admissibility of scientific and technical evidence at trial. They do not currently control judicial reliance on scientific and technical evidence in probable cause hearings.

[75] For instance, imagine that an entity claiming privilege has chosen to store information in a manner that makes redacting the privileged information exceedingly difficult, but could feasibly have stored the information in an alternative way to make redaction easy. In such a case, a court considering forwards-looking feasibility might look unfavorably on the privilege claim.

[76] *Cf.* Matter of Leopold to Unseal Certain Electronic Surveillance Applications and Orders, 300 F. Supp. 3d 61, 93-97 (D.D.C. 2018) (ordering similar prospective relief).

[9] Berman, M. Most police departments in america are small. that's partly why changing policing is difficult, experts say. Accessed: 2022-02-01. URL: www.washingtonpost.com/nation/2021/05/08/most-police-departments-america-are-small-thats-partly-why-changing-policing-is-difficult-experts-say/.

[10] Bissias, G., Levine, B., Liberatore, M., Lynn, B., Moore, J., Wallach, H., and Wolak, J. Characterization of contact offenders and child exploitation material trafficking on five peer-to-peer networks. *Child abuse & neglect 52* (2016), 185–199.

[11] Blum, M., Feldman, P., and Micali, S. Non-interactive zero-knowledge and its applications. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. 2019, pp. 329–349.

[12] Brassard, G., Chaum, D., and Crépeau, C. Minimum disclosure proofs of knowledge. *Journal of computer and system sciences 37*, 2 (1988), 156–189.

[13] Calo, R. Robotics and the lessons of cyberlaw. *Calif. L. Rev. 103* (2015), 513.

[14] Canetti, R., Kilian, J., Petrank, E., and Rosen, A. Black-box concurrent zero-knowledge requires\tilde {Ω}(log n) rounds. In *Proceedings of the thirty-third annual ACM symposium on Theory of computing* (2001), pp. 570–579.

[15] Crootof, R., and Ard, B. Structuring techlaw. *Harvard Journal of Law & Technology 34* (2021), 347–417.

[16] Dwork, C., Naor, M., and Sahai, A. Concurrent zero-knowledge. *Journal of the ACM (JACM) 51*, 6 (2004), 851–898.

[17] EveryCRSReport. Federal power over local law enforcement reform: Legal issues. Accessed: 2022-02-01. URL: www.everycrsreport.com/reports/R44104.html.

[18] Feige, U., Lapidot, D., and Shamir, A. Multiple non-interactive zero knowledge proofs based on a single random string. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science* (1990), IEEE, pp. 308–317.

[19] Fiat, A., and Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques* (1986), Springer, pp. 186–194.

[20] Goldberg, L., Papini, S., and Riabzev, M. Cairo–a turing-complete stark-friendly cpu architecture. *Cryptology ePrint Archive* (2021).

[21] Goldreich, O., and Krawczyk, H. On the composition of zero-knowledge proof systems. *SIAM Journal on Computing 25*, 1 (1996), 169–192.

[22] Goldreich, O., Micali, S., and Wigderson, A. Proofs that yield nothing but their validity or all languages in np have zero-knowledge proof systems. *Journal of the ACM (JACM) 38*, 3 (1991), 690–728.

[23] Goldreich, O., and Oren, Y. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology 7*, 1 (1994), 1–32.

[24] Goldwasser, S., Micali, S., and Rackoff, C. The knowledge complexity of interactive proof systems. *SIAM Journal on computing 18*, 1 (1989), 186–208.

[25] Grinman, A. J. *Natural language processing on encrypted patient data*. PhD thesis, Massachusetts Institute of Technology, 2016.

[26] Groth, J., Ostrovsky, R., and Sahai, A. Perfect non-interactive zero knowledge for np. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques* (2006), Springer, pp. 339–358.

[27] Groth, J., Ostrovsky, R., and Sahai, A. New techniques for noninteractive zero-knowledge. *Journal of the ACM (JACM) 59*, 3 (2012), 1–35.

[28] Groth, J., and Sahai, A. Efficient non-interactive proof systems for bilinear groups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques* (2008), Springer, pp. 415–432.

[29] Haasz, A. Underneath it all: Policing international child pornography on the dark web. *Syracuse J. Int'l L. & Com. 43* (2015), 353.

[30] Holt, T. J., Cale, J., Leclerc, B., and Drew, J. Assessing the challenges affecting the investigative methods to combat online child exploitation material offenses. *Aggression and violent behavior* (2020), 101464.

[31] Hyland, S. S., and Davis, E. Local police departments, 2016: Personnel. Accessed: 2022-02-01. URL: www.bjs.ojp.gov/content/pub/pdf/lpd16p.pdf.

[32] Imwinkelried, E. J. The new wigmore: An essay on rethinking the foundation of evidentiary privileges. *BUL Rev. 83* (2003), 315.

[33] Imwinkelried, E. J., and Garland, N. M. *Exculpatory Evidence: The Accused's Constitutional Right to Introduce Favorable Evidence*. LexisNexis, 2020.

[34] Koningisor, C. Secrecy creep. *U. Pa. L. Rev. 169* (2020), 1751.

[35] Kroll, J. A. *Accountable algorithms*. PhD thesis, Princeton University, 2015.

[36] Kroll, J. A. *Accountable algorithms*. PhD thesis, Princeton University, 2015.

[37] Leclerc, B., Drew, J., Holt, T., Cale, J., and Singh, S. *Child sexual abuse material on the darknet: a script analysis of how offenders operate*. No. Trends and issues in crime and criminal justice, no. 627. Australian Institute of Criminology, 2021.

[38] Liberatore, M., Erdely, R., Kerle, T., Levine, B. N., and Shields, C. Forensic investigation of peer-to-peer file sharing networks. *digital investigation 7* (2010), S95–S103.

[39] Manes, J. Secrecy & evasion in police surveillance technology. *Berkeley Tech. LJ 34* (2019), 503.

[40] Moses, L. B. Why have a theory of law and technological change. *Minn. JL Sci. & Tech. 8* (2007), 589.

[41] Nakashima, E. This is how the government is catching people who use child porn sites, 2016. Accessed: 2022-02-01. URL: https://www.washingtonpost.com/world/national-security/how-the-government-is-using-malware-to-ensnare-child-porn-users/2016/01/21.

[42] Parameswaran, M., Susarla, A., and Whinston, A. B. P2p networking: an information sharing alternative. *Computer 34*, 7 (2001), 31–38.

[43] Peersman, C., Schulze, C., Rashid, A., Brennan, M., and Fischer, C. icop: Live forensics to reveal previously unknown criminal media on p2p networks. *Digital Investigation 18* (2016), 50–64.

[44] Pouwelse, J., Garbacki, P., Epema, D., and Sips, H. The bittorrent p2p file-sharing system: Measurements and analysis. In *International workshop on peer-to-peer systems* (2005), Springer, pp. 205–216.

[45] Shamir, A. Ip= pspace. *Journal of the ACM (JACM) 39*, 4 (1992), 869–877.

[46] Sinnar, S. Procedural experimentation and national security in the courts. *California Law Review 106*, 4 (2018), 991–1060.

[47] Smith, S. W. Policing hoover's ghost: the privilege for law enforcement techniques. *Am. Crim. L. Rev. 54* (2017), 233.

[48] StarkWare. Starkware solution architecture, 2022. Accessed: 2022-02-01. URL: www.docs.starkware.co/starkex-v4/architecture/solution-architecture.

[49] StarkWare. Using sharp, 2022. Accessed: 2022-02-01. URL: www.cairo-lang.org/docs/sharp.html.

[50] Wallach, H. M. Topic modeling: beyond bag-of-words. In *Proceedings of the 23rd international conference on Machine learning* (2006), pp. 977–984.

[51] Wang, C., and Li, B. Peer-to-peer overlay networks: A survey. *Department of Computer Science, The Hong Kong University of Science and Technology, Hong Kong* (2003), 9.

[52] Watch, H. R. Secret origins of evidence in us criminal cases. Accessed: 2022-02-01. URL: www.hrw.org/report/2018/01/09/dark-side/secret-origins-evidence-us-criminal-cases.

[53] Weidman, M. Jurisdiction, the internet, and the good faith exception: Controversy over the government's use of network investigative techniques. *Dickinson L. Rev. 122* (2017), 967.

[54] Westlake, B. G. The past, present, and future of online child sexual exploitation: Summarizing the evolution of production, distribution, and detection. *The Palgrave handbook of international cybercrime and cyberdeviance* (2020), 1225–1253.

[55] Wexler, R. Privacy asymmetries: Access to data in criminal defense investigations. *UCLA L. Rev. 68* (2021), 212.

[56] Wikipedia. Bittorrent, 2022. Accessed: 2022-02-01. URL: www.en.wikipedia.org/wiki/BitTorrent.

[57] Zhang, Y., Jin, R., and Zhou, Z.-H. Understanding bag-of-words model: a statistical framework. *International Journal of Machine Learning and Cybernetics 1*, 1 (2010), 43–52.

A POSSIBLE GENERALIZATIONS

Most parts of this article focus on a particular instance of the hidden investigative software problem. We have described that instance in detail, presented an implementation of a proof-of-concept solution for that instance, and analyzed the legal implications of that solution. Taking a more general perspective, the methods and algorithms we developed to tackle that particular instance may also be used to address other instances of the hidden investigative software problem, as well as other types of verification dilemmas. This section elaborates on that possibility.

Interactions between parties often reach a crossroads at which, to proceed with the interaction, one party needs to substantiate some facts to the other party. It is often the case that the most straightforward (or only seen) way of substantiating these facts involves the disclosure of information beyond the mere facts sought to be verified. In such scenarios, the first party must choose between two costly options. One option is to substantiate the facts via disclosing the relevant information, which may be problematic (or even unacceptable) due to privacy or secrecy concerns. Another option is not to substantiate the facts and forgo the opportunity. As mentioned above, such problems are called verification dilemmas (also discussed in [3]). Prior literature has identified verification dilemmas across a series of legal domains, including scenarios where individuals must reveal private information in order to verify identity; companies must reveal business secrets in order to negotiate a deal; governments must reveal sensitive data and algorithms in order to enable public oversight, and; litigants pursuing trade secret misappropriation claims in court must reveal their trade secrets to vindicate their legal rights [3, 36]. In some of these cases, the secret piece of information that stands at the core of the verification dilemma is a computer text file (or can be readily put in the form of a text file). Such problems are *text-based verification dilemmas*

In any of these cases, if an acceptable solution is that a trusted party inspects the secret text file, extracts answers to a list of queries agreed upon by the prover and verifier, and reports back the answers to these queries, then it is possible to use our text-processing tools to address that verification dilemma without employing the trusted third party. We suggest the following approach to do that.

Commitment. The prover runs an Initializer, like the one presented in Section 3, over the secret text file and obtains an object that can be fed into a ZKP-friendly programming language (e.g., convert to a list of Cairo felts), denoted X, a cryptographic commitment, denoted $H(X)$, and sends $H(X)$ to the verifier.

Design check program. Assuming the prover and verifier agree on the queries to be checked over the text file, they use text-processing tools written in that ZKP-friendly programming language (like our Cairo text-processing programs) to code a check program that implements these queries. That check program gets X as a secret input, computes the hash of its input, extracts answers to the agreed queries, and outputs the answers to these queries along with the hash of the input.

Execute. The prover runs the check program over X and obtains an output, denoted Z. We stress that, by design, Z includes the hash of the input.

Prove. The prover executes a ZKP proof-generation algorithm (e.g., the [7] proof-generation algorithm – see 70) to generate a ZK-proof P attesting to the integrity of the execution of the check program over X. The prover then sends P to the verifier, along with the output of the execution of the check program over X.

Verify. The verifier verifies P (e.g., using the [7] verification algorithm). Then, the verifier asserts that the hash of the check program's input is identical to the hash received from the prover at the commitment stage.

We note that implementing such a solution for other verification dilemmas may be much simpler than solving the particular verification dilemma we addressed in this paper - the hidden investigative software problem. The reason for that is that verifying attributes of a program by querying the source code may be hard (or impossible) in some cases. It requires designing the program components appropriately and adequately defining the list of text-queries to identify these attributes. On the contrary, some of the verification dilemmas described in [3] are easy to address using our system. Demonstrating that is left for future work.

One scenario where the suggested workflow may be adopted is verifying the authenticity of redacted documents. In some legal cases, the prosecution may disclose redacted or excerpted files to the defense. We stress that the possible reasons for redacting documents are distinct legal issues from the law enforcement evidentiary privilege. This article neither discusses nor endorses alternative rationales for redaction beyond the law enforcement evidentiary privilege. However, if redacted documents are presented, the defense may use the suggested workflow to verify that the redacted files are 100% authentic and complete.

Bag-of-words. Our Cairo text-processing tools, presented in Section 3, can be used various types of advanced search and examination of text files. On top of these tools, we also developed programs for computing the bag-of-words (BOW) of a given text file and related tasks. These programs may be used in scenarios where the queries that the verifier wants to verify are more complicated, and relate to the BOW of the text file. A description of these programs appears below.

`Count_w_in_x`. This function gets a text file `X` and a word `w`, checks how many times `w` appears in `X`, and returns the number of appearances. Its main difference from `Count_arr_in_x` is that it gets a single word (instead of an array of such).

`Bow`. This function gets a text file `X` and returns its BOW. I.e., it returns two arrays – `seen` and `counts`. The first array contains all the distinct words that appear in `X`, and the second contains their corresponding number of appearances.

`Bag_of_2grams`. This function gets a text file `X` and returns its bag-of-2grams. I.e., it returns two arrays. The first array contains all the distinct pairs of words that appear adjacently in `X`, and the second contains their corresponding number of appearances.

`Sum_bags`. This function gets two BOWs and returns their sum. I.e., it gets two `seen` arrays named s_1 and s_2, and two `counts` arrays named c_1 and c_2, and returns two arrays. The first contains all the words that appear either in s_1 or s_2, and the second contains the sum of their corresponding counts from c_1 and c_2.